

Investigating Performance Bottlenecks in the Plan 9 SPI Driver

Nick Ingravallo

> **whoami**

- Computer Science major at Drexel
- Involved in Plan 9 through Professor Brian Stuart
 - Summer '22 - ENIAC Research

The ENIAC Simulator



<https://www.cs.drexel.edu/~bls96/eniac/eniacrun-s.jpg>

The Plan 9 Research

- Emailed Professor Stuart about Operating Research ~this time last year
- He mentioned he had some Plan 9 related work...

P9 ENIAC Simulator

- ENIAC Simulator console running on Plan 9 with LED matrix
- The LED matrix is flashing!
- Some kind of issue in the real-time scheduler?

Testing the Real-Time Scheduler

1. Buy a Raspberry Pi 4
 - a. Multi-core machine
2. To simulate system load, a `while(1)` loop and pinned to each core
3. Ran a 30,000 x 1ms real-time loop to test

```
pid = getpid();  
path = smprint("/proc/%d/ctl", pid);  
cfd = open(path, ORDWR);  
fprintf(cfd, "wired %d\n", core);  
fprintf(cfd, "period 1ms\n");  
fprintf(cfd, "cost 1ms\n");  
  
for (i = 0; i < 30000; i++) {  
    sleep(0);  
}  
print("done\n");
```

*A 30000 * 1ms loop, ~30 second time expected*

Results

	Core 0
Time (seconds)	30.0x

.0x meaning results within .01s-.05s

Timed using `time [program]`

Results

	Core 0	Core 1
Time (seconds)	30.0x	30.0x

.0x meaning results within .01s-.05s

Timed using `time [program]`

Results

	Core 0	Core 1	Core 2	Core 3
Time (seconds)	30.0x	30.0x	30.0x	30.0x

.0x meaning results within .01s-.05s

Timed using `time [program]`

The Good and Bad News

The Good News:

- Real-time scheduler operates as expected!

The Bad News:

- We still don't know what is failing...

Next Steps to Find the Culprit

- The ENIAC Simulator engages GPIO and SPI
- Possibly a failure in one of those drivers?
- Let's write C code to test!

```
fdg = open("#G/gpio", OWRITE);
fprintf(cfd, "period 1ms\n");
fprintf(cfd, "cost 1ms\n");
admitVal = fprintf(cfd, "admit\n");
print("admit val - %d\n", admitVal);

spictl = open("#pi/spictl", ORDWR);
spidat = open("#pi/spi0", ORDWR);

memset(buf, 0x41, 8);
for (i = 0; i < 30000; i++) {
    fprintf(fdg, "set 25 1\n");
    fprintf(fdg, "set 25 1\n");
    fprintf(fdg, "set 25 1\n");
    fprintf(fdg, "set 25 1\n");
    fprintf(fdg, "set 25 1\n");
    fprintf(fdg, "set 25 1\n");
    fprintf(fdg, "set 25 1\n");
    fprintf(fdg, "set 25 1\n");
    fprintf(fdg, "set 25 1\n");
    pwrite(spidat, buf, 8, 0);
    sleep(0);
}
print("done\n");
```

Invoking SPI / GPIO in realtime 30s loop

Code snippet partially from ledmat.c, Brian Stuart

Results (GPIO + SPI)

	Core 0
Time (seconds)	30.0x

.0x meaning results within .01s-.05s

Timed using `time [program]`

Results (GPIO + SPI)

	Core 0	Core 1
Time (seconds)	30.0x	69.68

.0x meaning results within .01s-.05s

Timed using `time [program]`

Results (GPIO + SPI)

	Core 0	Core 1	Core 2	Core 3
Time (seconds)	30.0x	69.68	69.6x	69.6x

.0x meaning results within .01s-.05s

Timed using `time [program]`

Interpreting the Results

- Either the SPI driver, GPIO driver, or both caused delays...
- We're going to need to test all three scenarios!

The Delays Caused By...

(It was the SPI driver)

```
spictl = open("#pi/spictl", ORDWR);
spidat = open("#pi/spi0", ORDWR);

memset(buf, 0x41, 8);
for (i = 0; i < 30000; i++) {
    pwrite(spdat, buf, 8, 0);
    sleep(0);
}
print("done\n");
```

SPI driver code isolated

Results (SPI)

	Core 0	Core 1	Core 2	Core 3
Time (seconds)	30.0x	69.6x	69.6x	69.6x

.0x meaning results within .01s-.05s

Timed using `time [program]`

The SPI Driver Code

- The SPI read/write utilizes Direct Memory Access (DMA)
- `dmastart()` called twice, `dmawait()` once
- Is there a faster solution?

```

1 void
2 spirw(uint cs, void *buf, int len)
3 {
4     Spiregs *r;
5
6     assert(cs <= 2);
7     assert(len < (1<<16));
8     qlock(&spi.lock);
9     if(waserror()){
10         qunlock(&spi.lock);
11         nexterror();
12     }
13     if(spi.regs == 0)
14         spiinit();
15     r = spi.regs;
16     r->dlen = len;
17     r->cs = (r->cs & (Cpha | Cpol)) | (cs << Csshift) | Rxclear | Txclear | Dmaen | Adcs | Ta;
18     /*
19      * Start write channel before read channel - cache wb before inv
20      */
21     dmastart(DmaChanSpiTx, DmaDevSpiTx, DmaM2D,
22             buf, &r->data, len);
23     dmastart(DmaChanSpiRx, DmaDevSpiRx, DmaD2M,
24             &r->data, buf, len);
25     if(dmawait(DmaChanSpiRx) < 0)
26         error(Eio);
27     cachedinvse(buf, len);
28     r->cs &= (Cpha | Cpol);
29     qunlock(&spi.lock);
30     poperror();
31 }

```

```
8      /*
9      * Start write channel before read channel - cache wb before inv
10     */
11     dmastart(DmaChanSpiTx, DmaDevSpiTx, DmaM2D,
12             buf, &r->data, len);
13     dmastart(DmaChanSpiRx, DmaDevSpiRx, DmaD2M,
14             &r->data, buf, len);
15     if(dmawait(DmaChanSpiRx) < 0)
16         error(Eio);
17     cachedinvse(buf, len);
```

Snippet from spirw() function

DMAM2D - DMA Memory 2 Device

DMAD2M - DMA Device 2 Memory

cachedinvse() - cache invalidate

Any faster solutions?

- Direct memory polling may be faster
 - Alternative to DMA, but not concurrent


```
void spirw(uint cs, void *buf, int len){
    int i;
    // ... initialization code unchanged ...
    spi.regs->cs = Rxclear | Txclear | Adcs | Ta;
    for (i = 0; i < len; i++) {
        spi.regs->data = ((char*)buf)[i];
        while ((spi.regs->cs & Txd) == 0);
    }
    while ((spi.regs->cs & Done) == 0);
    spi.regs->cs = 0;
    qunlock(&spi.lock);
    poperror();
}
```

spirw() with direct polling

```
spictl = open("#pi/spictl", ORDWR);
spidat = open("#pi/spi0", ORDWR);

memset(buf, 0x41, 8);
for (i = 0; i < 30000; i++) {
    pwrite(spdat, buf, 8, 0);
    sleep(0);
}
print("done\n");
```

SPI driver code isolated

Results (SPI Polling vs SPI DMA)

DMA	Core 0	Core 1	Core 2	Core 3
Time (seconds)	30.0x	69.6x	69.6x	69.6x

.0x meaning results within .01s-.05s

Timed using `time [program]`

Results (SPI Polling vs SPI DMA)

Polling	Core 0
Time (seconds)	30.0x

DMA	Core 0	Core 1	Core 2	Core 3
Time (seconds)	30.0x	69.6x	69.6x	69.6x

.0x meaning results within .01s-.05s

Timed using `time [program]`

Results (SPI Polling vs SPI DMA)

Polling	Core 0	Core 1	Core 2	Core 3
Time (seconds)	30.0x	30.0x	30.0x	30.0x

DMA	Core 0	Core 1	Core 2	Core 3
Time (seconds)	30.0x	69.6x	69.6x	69.6x

.0x meaning results within .01s-.05s

Timed using `time [program]`

Some Hidden Advantages

- We can transfer more data with polling in real-time!
- DMA Max - 0x24 Bytes
- Polling Max - 0x40 Bytes
- ~75% more data, complete drop off after max
- But why is polling more performant?

Why is Polling More Performant?

- There were ~4 more weeks left in our research term
 - Could we find the root issue?
- What caused the ~133% spike in runtime?

Let's Look at System Interrupts

- Core 0 is assigned all peripheral interrupts
 - (IRQ # ≥ 32)
- Non-0 cores assigned inter-processor interrupts
 - (IRQ # < 32)


```

Nsgi = 16,          /* software-generated (inter-processor) intrs */
Nppi = 32,          /* sgis + other private peripheral intrs */
...
...
if (m->machno == 0) {
    cpumask = 1<<0;          /* just cpu 0 */
    for (i = Nppi; i < sizeof idp->targ; i++)
        idp->targ[i] = cpumask;
    ...
}
else {
    ...
    for (i = 0; i < Nppi; i++)
        idp->targ[i] = 1<<machno;
    ...
}

```

src/9/bcm/trap4.c

The DMA Interrupts

- The interrupt request number for DMA driver is $> \#96$
- IRQs $> \#96$ are considered “Global Interrupts”
- Global interrupts are handled on core 0

What Does This Mean?

- If we start a program that interfaces with SPI on core 1, DMA is initialized on the same core
- Upon the call to `dmastart()` in `spirw()`, the DMA interrupt is still handled on core 0
- A cross-core wakeup is required on each call!

Conclusion

- Polling solutions requires no calls to DMA driver
- Concurrency benefits are negated by slow interrupts
- Performance degradation not detected with polling
- Clearly the ideal patch!
 - *Or...*

**Can we get around the cross-core
interrupts?**

intrto(int cpu, int irq)

- src/9/bcm/trap4.c
- We can re-route an interrupt to a specific core
- What if we change the DMA interrupt to its caller's core?
- Conveniently, dmastart() contains the caller's core

```
// dma.c
166      ...
167      intrenable(IRQDMA(chan), dmainerrupt, ctrl, 0, "dma");
168  }
->
->  intrto(m->machno, (IRQDMA(chan) + 64 + 32));
->
169  cb = ctrl->cb;
170  ti = 0;
171  switch(dir){
    ...
```

src/9/bcm/dma.c

```
1 void
2 intrto(int cpu, int irq)
3 {
4     Intrdistregs *idp = (Intrdistregs *) (ARMLocal+Intrdist);
5
6     /* first Nppi are read-only for SGIs and the like */
7     ilock(&distlock);
8     idp->targ[irq] = 1 << cpu;
9     iunlock(&distlock);
10 }
11
```

src/9/bcm/trap4.c

Results From `intrto()`

- No performance degradation
- SPI code untouched
- The rest of the system performed identically in preliminary testing

Thank You!



Original Write-Up (QR)
<https://plan9.nickv.ing/>